

# GPU-Accelerated TEBD for the 1D Transverse-Field Ising Model

Final Report, CME 213: Parallel Computing with CUDA, MPI and OpenMP

Hercy Shen, Chiling Han, Xin Wei — June 8, 2026

## 1. Project description

We simulate the **real-time quench dynamics** of the one-dimensional transverse-field Ising model (TFIM), a canonical quantum many-body system, with Hamiltonian

$$H = -J \sum_i \sigma_i^x \sigma_{i+1}^x - g \sum_i \sigma_i^z. \quad (1)$$

A chain of  $L$  spin- $\frac{1}{2}$  sites lives in a Hilbert space of dimension  $2^L$ , so storing the wavefunction exactly is impossible beyond  $L \approx 30$ . What makes this tractable is that low-energy and short-time states of 1D local Hamiltonians stay weakly entangled, so they compress into a *Matrix Product State (MPS)*: the  $2^L$  amplitudes factor into  $L$  small rank-3 tensors connected by *bond* indices of dimension  $\chi$ . The MPS stores only  $O(L\chi^2d)$  numbers ( $d = 2$ ), where  $\chi$  grows with the entanglement across each cut.

**Computation.** We evolve the state with *Time-Evolving Block Decimation (TEBD)*: the propagator  $e^{-iHt}$  is Suzuki–Trotter split into nearest-neighbour two-site gates  $U_b = e^{-i\Delta t H_b}$ , and each Trotter step applies  $L - 1$  gates. Applying one gate is one small complex SVD that re-compresses the two-site tensor back to bond dimension  $\chi$ ; this SVD takes  $> 99.6\%$  of GPU kernel time and is the target of all parallel work here.

- **Inputs:** chain length  $L$ ; couplings  $(J, g)$ ; an initial product state; Trotter step  $dt$ ; number of steps  $N$ ; max bond dimension  $\chi_{\max}$ .
- **Outputs:** mean field magnetizations  $\langle \sigma_i^z \rangle$ ,  $\langle \sigma_i^x \rangle$ , two-point correlations  $\langle \sigma_i^x \sigma_j^x \rangle$ , the bipartite von-Neumann entanglement entropy, and the bond-dimension profile  $\chi(t)$ .
- **Assumptions:** open boundary conditions, local dimension  $d = 2$ .

**Why parallelism.** Entanglement grows under a quench, so  $\chi(t)$  climbs over time (Fig. 3) until it saturates  $\chi_{\max}$ . Each bond update requires a small dense SVD whose cost scales roughly as  $O(\chi^3)$ , so doubling the effective bond dimension gives an  $8\times$  increase in work.

Across GPUs we use task parallelism: each MPI rank owns one GPU and runs independent TEBD evolutions for different  $(J, g)$  points – a natural fit for the parameter sweep.

## 2. Algorithms and state of the art

**Core algorithm.** We implement TEBD on a *right-canonical MPS* in Vidal form. The two-site update is: (1) contract the gate with the two-site tensor; (2) SVD the reshaped  $(\chi_L d) \times (d \chi_R)$  matrix; (3) truncate to the largest  $\chi_{\max}$  singular values and renormalize; (4) restore MPS form.

**State of the art.** Mature libraries (*TeNPy*, *ITensor*, *quimb*, *TensorNetwork*) are CPU/Python-centric and general; on GPUs, NVIDIA’s *cuTensorNet* (cuQuantum) offers library-level batched contraction and SVD. The algorithmic variants relevant to our problem are the SVD engine, the time integrator (Trotter–Suzuki), and the parallel decomposition.

**Our choice and how it compares.** We do not try to beat cuTensorNet on throughput; instead we build an instrumented TEBD that times each stage separately, so we can attribute time to gate contraction, SVD, truncation, and PCIe traffic rather than let a library hide it.

### 3. Parallelization strategy (CUDA + MPI)

#### 3.1. The central difficulty and our decomposition

A single TEBD chain has *limited spatial parallelism*. Within one Trotter sub-step the even bonds  $\{0, 2, 4, \dots\}$  and odd bonds  $\{1, 3, 5, \dots\}$  touch disjoint tensors and are independent ( $\sim (L-1)/2$ -way), but the even and odd sub-steps are *sequential*, and a spatial decomposition that splits the chain across ranks would require one MPI round-trip per sub-step to exchange the boundary Schmidt spectrum. We benchmark up to  $L = 80$  and found that the per-step compute is far too small to hide that latency.

We therefore use *task parallelism*: each MPI rank owns one GPU and runs complete, independent TEBD evolutions for a set of  $(J, g)$  points in the phase diagram, with zero in-loop communication. We rejected two alternatives: (a) spatial chain decomposition, infeasible at small  $L$  due to per-sub-step synchronization; (b) replicated-parameter plus distributed chain, which needs per-bond MPI barriers.

#### 3.2. Single-GPU bond-update pipeline

The bond update runs as a five-stage device pipeline – gate contraction (gate staged in **shared memory**), a **coalesced** row $\rightarrow$ col relay layout, the SVD ( $> 99.6\%$  of kernel time, §5), and two Vidal-restore steps (full kernel list in Appendix B).

#### 3.3. Persistent device-resident MPS and streams

- **Persistent device MPS.** All tensors and Schmidt spectra stay device-resident for the whole evolution, and the buffers are pre-allocated once, so nothing is allocated during the evolution itself.
- **Streams within a sub-step.** A sub-step can run several workspaces on independent CUDA streams. But truncation needs a host sync mid-bond to read the singular values, which serializes the bonds, so the measured stream speedup is negligible (Table 1).

#### 3.4. MPI decomposition and communication pattern

Each rank calls `cudaSetDevice(rank % n_gpus)`, builds its  $(J, g)$  points locally (no scatter), and runs independent evolutions; the TEBD loop contains no MPI calls. Only at the end does rank 0 collect results: an `MPI_Gather` of per-rank record counts, then an `MPI_Gatherv` of the variable-length records.

### 4. Performance model

The performance model has four parts, each with a falsifiable prediction.

**(a) Per-bond compute and the latency/compute crossover.** The dominant cost is the SVD of a  $2\chi_a \times 2\chi_a$  complex matrix. One-sided Jacobi does  $O(\chi_a^3)$  flops per sweep over  $\sim 5$ –10 sweeps, plus a fixed overhead  $t_0$  per bond, giving  $T_{\text{step}}(L, \chi_a) \approx (L-1)(t_0 + c\chi_a^3)$ . **Predictions:** (i) when  $c\chi_a^3 \ll t_0$  the code is **latency-bound**:  $T$  is linear in  $L$  and flat in  $\chi_{\text{max}}$ ; (ii) when  $c\chi_a^3 \gg t_0$  it becomes **work-dominated**:  $T \propto \chi_a^3$ . Our runs sit in branch (i) (§6).

**(b) Roofline / arithmetic intensity.** Each Jacobi sweep does  $O(\chi_a^3)$  flops on an  $O(\chi_a^2)$ -word matrix that fits in L2, so intensity  $I = O(\chi_a)$  grows with  $\chi_a$  and the matrix is cache-resident. **Prediction:** the SVD is never bandwidth-bound; at the small realized  $\chi_a$  the limiter is kernel-launch and dispatch latency, not bandwidth or peak compute, and the compute roof is reached only at  $\chi_a$  far beyond our range.

**(c) Communication model.** The only message is the final `Gatherv` of  $\lesssim 10$  KB. **Prediction:** communication fraction  $< 10^{-4}$ , invisible in a profiler.

(d) **Parallel efficiency (task-parallel load balance).** With  $P$  ranks and per-pair costs  $t_i$ , efficiency is  $E = (\sum_i t_i) / (P \max_r \sum_{i \in r} t_i)$ . Pair cost rises steeply with  $J$ : more entanglement means larger  $\chi_a$ , and the SVD work grows as  $O(\chi_a^3)$ .

## 5. Benchmarking and instrumentation

**Instrumentation.** Per-stage GPU time measured with *CUDA events* (`bench_breakdown`); full-step wall time via host timers (`bench_gpu`); SVD-engine comparison via `bench_svd`; end-to-end timelines with *Nsight Systems* (`nsys profile`), single-GPU and under `mpirun -n 4`.

**Single-GPU full-step timing** (`bench_gpu`,  $L = 20$ , 20 steps, `gpu-turing`):

| $\chi_{\max}$ | CPU (s) | M3 GPU (s) | Persistent (s) | Streamed (s) | CPU/Persistent |
|---------------|---------|------------|----------------|--------------|----------------|
| 16            | 2.049   | 5.372      | 5.180          | 5.183        | 0.40           |
| 32            | 9.601   | 13.196     | 12.992         | 13.064       | 0.74           |
| 48            | 20.073  | 17.428     | 17.075         | 17.079       | 1.18           |
| 64            | 24.099  | 18.250     | 17.709         | 17.745       | 1.36           |
| 96            | 24.484  | 18.269     | 17.722         | 17.854       | 1.38           |
| 128           | 24.533  | 18.334     | 17.848         | 17.853       | 1.37           |

**Table 1:** GPU overtakes CPU at  $\chi_{\max} \geq 48$ ; persistent MPS is 1–3% faster than M3 host-staging; four streams add nothing.

A per-kernel breakdown (`bench_breakdown` / `Nsight cuda_gpu_kern_sum`) confirms the SVD dominates: cuSOLVER Jacobi is  $> 99.6\%$  of GPU kernel time, all our custom kernels together  $< 0.4\%$ , and MPI  $< 0.01\%$  (Table 4).

**Model vs. measurement.** The crossover study (Fig. 4) confirms prediction (a): at TFIM-realized  $\chi_a \approx 41$  the GPU is latency-bound (flat across  $\chi_{\max} = 64$ –128 in Table 1), crossing the CPU near  $\chi \approx 96$  and reaching  $\sim 5\times$  at  $\chi = 128$ . Nsight confirms model (c): no MPI call appears in the timeline.

## 6. Bottleneck analysis

The bottleneck has three parts, each tied to hardware.

- **Not memory-bound.** The SVD is  $> 99.6\%$  of kernel time (Table 4), and roofline rules out bandwidth: the  $L_2$ -resident  $2\chi_a \times 2\chi_a$  matrix has intensity  $\sim 95\times$  past the ridge at  $\chi_a = 64$ . But that only rules out bandwidth – it does not show the kernel reaches the compute roof.
- **Latency-bound at the realized sizes.** Entanglement fixes the realized  $\chi_a \approx 41$  ( $\leq 64$  across the sweep), too small to saturate the SMs. The GPU/CPU ratio in Table 1 confirms this: the GPU loses at small  $\chi_{\max}$  (launch overhead  $t_0$  dominates), then wins only  $\sim 1.4\times$  and plateaus as  $\chi_a$  saturates – a compute-bound kernel would instead keep speeding up.
- **Multi-GPU: load imbalance, not communication.** Computation is  $> 99\%$  of wall time (`Gatherv`  $< 0.01\%$ ), so the distributed bottleneck is **load imbalance**: round-robin hands one rank the four hardest ( $J = 2.0$ ) pairs,  $\sim 2.2\times$  the cheapest, giving the  $\sim 19\%$  gap at  $P = 4$  (§8, Fig. 2).

**Custom-kernel profiling (Nsight Compute).** `ncu` on our hand-written Jacobi kernel (Appendix E) shows the same story at the kernel level: a single 128-thread block leaves 1 of 72 SMs active ( $\approx 99.8\%$  idle) and stalls 41.6% of the time at per-rotation `__syncthreads` barriers – under-decomposed and latency-bound, not compute- or memory-bound. cuSOLVER keeps far more of the device busy ( $1.2$ – $2.2\times$  faster, Table 2). This is the same shortage of per-chain parallelism behind our task-parallel design (§3): even batching the  $\sim (L - 1)/2$  bonds would not fill 72 SMs at  $L \leq 80$ .

## 7. Algorithmic variants and their effect on performance

We explored four variants; each shifts the bottleneck differently.

1. **Host-staging (M3)  $\rightarrow$  persistent device MPS.** Eliminating per-bond H2D/D2H cuts PCIe traffic from tens of MB to  $\sim 0.2$  MB for a steady 1–3% speedup (Table 1), small because the SVD dominates.
2. **Single stream  $\rightarrow$  4 streams.** Negligible (Table 1, Fig. 5). The model predicted it; profiling showed why: host-side truncation serializes bonds.
3. **cuSOLVER gesvdj vs. custom one-sided Jacobi SVD.** Our hand-written Jacobi kernel is **1.2–2.2 $\times$  slower** than cuSOLVER and agrees to  $\lesssim 10^{-12}$  (Table 2, Fig. 6). Both paths plateau once  $\chi_{\max}$  exceeds the realized  $\chi_a \approx 41$ –64 (cuSOLVER time changes  $< 1\%$  from  $\chi_{\max} = 64$  to 256), a direct consequence of the latency-bound regime: SVD matrix size is set by  $\chi_a$ , not  $\chi_{\max}$ .
4. **Trotter order (1/2/4).** Higher order trades more gates per step for  $O(dt^p)$  accuracy; order 4 (Forest–Ruth) reaches the ED-validation floor ( $\sim 10^{-6}$ ) at larger  $dt$ .

| $\chi_{\max}$ | cuSOLVER (s) | Custom Jacobi (s) | Ratio | max $ \Delta\sigma $   |
|---------------|--------------|-------------------|-------|------------------------|
| 16            | 7.110        | 8.184             | 0.869 | $4.74 \times 10^{-14}$ |
| 32            | 17.352       | 27.089            | 0.641 | $5.84 \times 10^{-13}$ |
| 48            | 22.688       | 46.056            | 0.493 | $9.96 \times 10^{-13}$ |
| 64            | 23.751       | 52.160            | 0.455 | $1.00 \times 10^{-12}$ |
| 128           | 23.754       | 52.208            | 0.455 | $1.00 \times 10^{-12}$ |
| 256           | 23.985       | 52.435            | 0.457 | $1.00 \times 10^{-12}$ |

**Table 2:** bench\_svd: cuSOLVER vs. custom Jacobi,  $L = 20$ , 20 steps. Both paths plateau above  $\chi_{\max} = 64$ ;

## 8. Scalability analysis

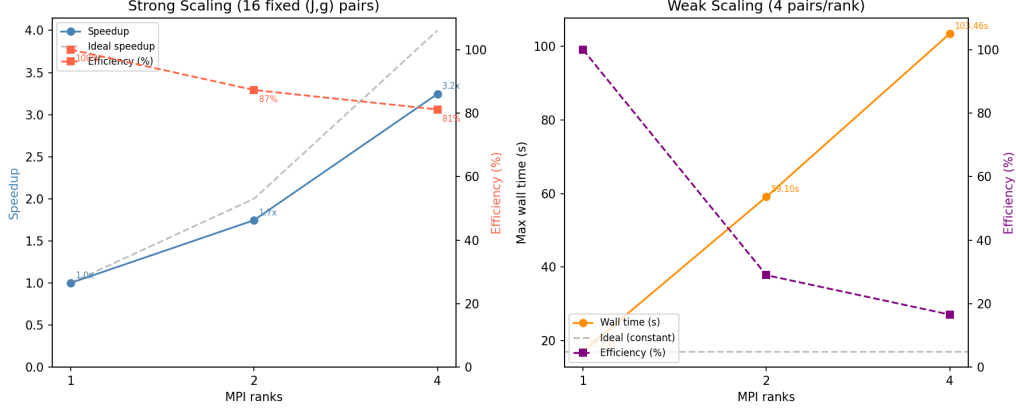
We study scaling of the task-parallel sweep on `gpu-turing`, using total rank wall time  $T(P) = \max_r \sum_{i \in r} t_i$ .

| Mode   | Ranks | Pairs/rank | Time (s) | Speedup | Efficiency |
|--------|-------|------------|----------|---------|------------|
| strong | 1     | 16         | 321.6    | 1.00    | 100%       |
| strong | 2     | 8          | 184.3    | 1.75    | 87%        |
| strong | 4     | 4          | 99.1     | 3.24    | 81%        |
| weak   | 1     | 4          | 17.1     | n/a     | 100%       |
| weak   | 2     | 4          | 59.1     | n/a     | 29%        |
| weak   | 4     | 4          | 103.5    | n/a     | 17%        |

**Table 3:** MPI strong and weak scaling. Strong speedup relative to  $P = 1$  wall time (321.6 s).

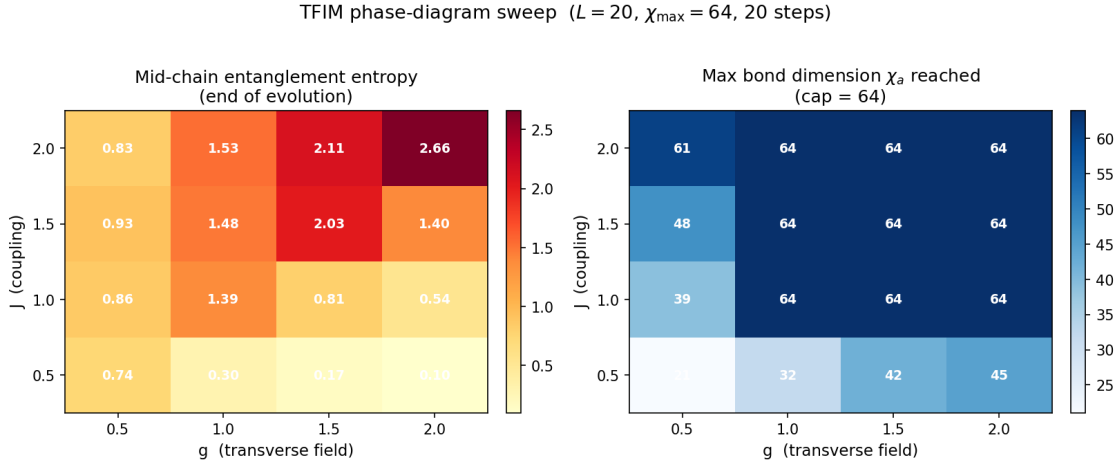
**Strong scaling.** Speedup reaches 3.24 $\times$  at  $P = 4$  (81% efficiency, Fig. 1). The 19% loss is exactly the load imbalance the model predicted: round-robin hands rank 3 the four  $J = 2.0$  pairs (highest entanglement), which run  $\sim 2.2\times$  longer than rank 0’s cheap  $J = 0.25$  pairs. Communication is  $< 0.01\%$ .

**Weak scaling.** Efficiency falls sharply (29% at  $P = 2$ , 17% at  $P = 4$ ), but this is *not* a communication or synchronization failure. The weak mode hands each added rank *higher- $J$*  pairs, which are intrinsically more expensive (2–6 $\times$  the cost of the  $P = 1$  pairs). True constant-work weak scaling would require balancing by *estimated entanglement* rather than pair count.



**Figure 1:** Strong-scaling speedup (left) and weak-scaling wall time (right) (companion to Table 3).

**Phase-diagram structure.** Figure 2 shows the end-of-evolution mid-chain entropy and realized  $\chi_a$  across the  $4 \times 4$  grid: entropy peaks near the critical point ( $J \approx g$ ) and saturates  $\chi_{\max} = 64$  at most high- $J$  points, while the paramagnetic corner reaches only  $\chi_a \approx 21$ –45. This 2–6 $\times$  spread in  $\chi_a$  sets the per-pair SVD cost and explains the load-imbalance gap.



**Figure 2:** Left: final mid-chain entropy  $S_{\text{mid}}$ . Right: max realized bond dimension  $\chi_a$ . High- $J$  points saturate  $\chi_{\max} = 64$  and dominate runtime; the paramagnetic corner (small  $J$ , large  $g$ ) is cheap.

## 9. Correctness and verification

We verify correctness at four independent levels.

- Unit tests.** 54 Google Test cases over 6 binaries cover the tensor and hand-written linear-algebra ops, observables, the TFI model, and TEBD invariants (norm,  $S^z$ , entropy growth, Trotter convergence), all passing at  $10^{-12}$  (exact) /  $10^{-10}$  (SVD-dependent).
- Exact-diagonalization oracle.** Against a full  $2^L \times 2^L$  evolution at  $L = 8$  (order 4,  $\chi_{\max} = 64$ ), TEBD reaches  $\sim 5 \times 10^{-6}$  max error in observables and entropy with fidelity loss  $4.3 \times 10^{-11}$ , dominated by the  $O(dt^4)$  Trotter floor (Fig. 8).
- CPU $\leftrightarrow$ GPU agreement.** GPU and CPU references agree to  $\lesssim 3.6 \times 10^{-13}$  over 30 steps – the floor where CPU Jacobi and cuSOLVER `gesvdj` match; custom Jacobi agrees to  $\lesssim 10^{-12}$  (Table 2).
- Multi-rank consistency.** 1-, 2-, and 4-rank sweeps produce identical  $(J, g) \rightarrow (\chi_{\max}, S_{\text{mid}})$

records, and 1- vs 4-stream configs agree to every printed digit.

## 10. Discussion, limitations, and future work

**What worked, and the largest payoffs.** Task-parallel MPI with zero in-loop communication gave 81% strong-scaling efficiency (load-imbalance-limited), and the persistent device MPS removed the M3 PCIe bottleneck.

**What was not worth the complexity.** *CUDA streams* gave negligible speedup (host truncation serializes bonds). Our from-scratch *custom Jacobi SVD* (the required hand-written kernel) is  $1.2\text{--}2.2\times$  slower than cuSOLVER, with `ncu` explaining why (§6).

### Limitations and future directions.

- **Load-balanced assignment.** Round-robin ignores the  $\sim 2\text{--}6\times$  spread across  $(J, g)$ ; sorting by estimated  $\chi_a$  and bin-packing should lift efficiency toward  $\gtrsim 95\%$  and fix the weak-scaling collapse.
- **Device-side truncation.** A GPU top- $k$  selection would remove the mid-pipeline host sync and let streams overlap bonds.
- **Spatial decomposition for  $L \gg 100$ .** Needed only when one evolution exceeds GPU memory or there are too few  $(J, g)$  points to fill the ranks; it would add an `MPI_Allgather` of the boundary spectrum per sub-step.

**Connection to lecture.** The project touches the GPU memory hierarchy (shared memory, coalesced layouts), the roofline model (the cache-resident SVD is never bandwidth-bound), the warp-execution model (the single-block Jacobi is latency/barrier-bound, not compute-bound), MPI collectives, and Amdahl/load-balance analysis.

## Appendix

### A. Hardware and software

Benchmarks ran on `gpu-turing` (Quadro RTX 6000, `sm_75`, CUDA 12.3, OpenMPI); development also used a local RTX 5090 (`sm_120`) and a one-core `g++` baseline.

### B. Bond-update pipeline

The single-bond update is a five-stage device pipeline. With  $m = \chi_L d$ ,  $n = d\chi_R$ ,  $K = \min(m, n)$ :

| # | Stage             | Kernel / call                                    | Notes                                                             |
|---|-------------------|--------------------------------------------------|-------------------------------------------------------------------|
| 1 | gate contraction  | <code>gate_contract_kernel&lt;2&gt;</code>       | gate in <b>shared memory</b> ; <code>#pragma unroll</code>        |
| 2 | row→col layout    | <code>row_to_col_kernel</code>                   | <b>coalesced</b> reads for column-major SVD                       |
| 3 | SVD               | <code>cusolverDnZgesvdj</code>                   | Jacobi, $K = \min(m, n)$ , <code>lwork</code> re-queried per call |
| 4 | Vidal restore (L) | <code>vidal_restore_left_kernel&lt;2&gt;</code>  |                                                                   |
| 5 | Vidal restore (R) | <code>vidal_restore_right_kernel&lt;2&gt;</code> |                                                                   |

Every stage is bracketed by CUDA events so `bench_breakdown` reports per-stage time.

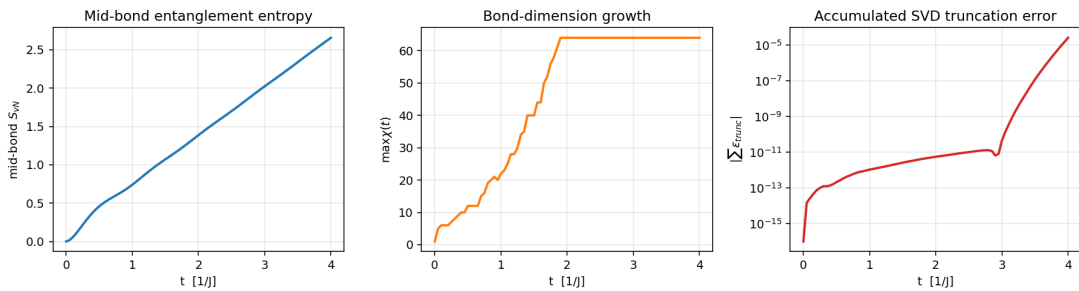
### C. Per-kernel time breakdown

Per-stage GPU kernel time (`bench_breakdown / Nsight cuda_gpu_kern_sum`), for the reference  $L = 20$  run:

| GPU kernel group                                                | Share   | Note                                  |
|-----------------------------------------------------------------|---------|---------------------------------------|
| cuSOLVER <code>gesvdbj</code> + <code>svd_col/row_rotate</code> | 96.4%   | top-3 Jacobi SVD kernels              |
| all remaining cuSOLVER helpers                                  | 3.2%    | sort, scale, QR, etc.                 |
| our CUDA kernels (gate contract, layout, Vidal restore)         | < 0.4%  | combined                              |
| <b>MPI communication</b>                                        | < 0.01% | <b>Gatherv</b> of $\sim 10$ KB at end |

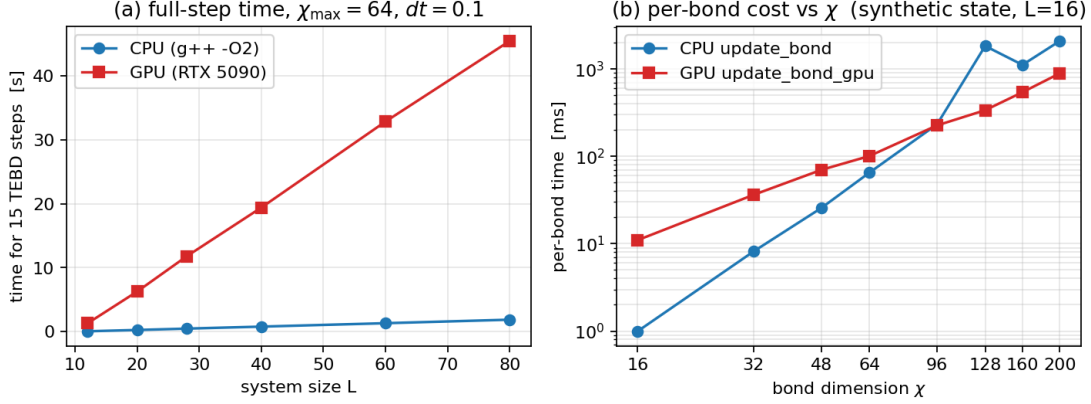
**Table 4:** cuSOLVER Jacobi is > 99.6% of GPU kernel time; all custom kernels together < 0.4%.

### D. Supplementary figures

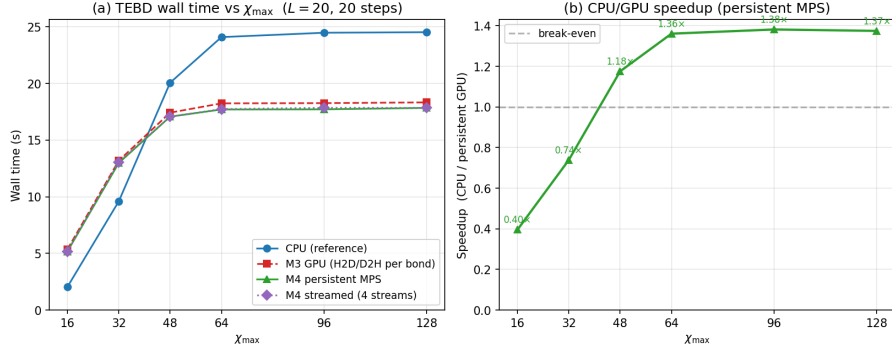


**Figure 3:** Reference  $L = 20$  TFIM quench ( $J = g = 1$ , order 4,  $\chi_{\max} = 64$ ). Entanglement growth drives bond-dimension growth, which sets the SVD sizes that dominate runtime.





**Figure 4:** (a) Wall time vs.  $L$  at  $\chi_{\max} = 64$ : linear,  $\sim 1.3$  ms/bond, latency-bound as predicted. (b) Per-call `update_bond` vs.  $\chi$ : CPU/GPU crossover near  $\chi \approx 96$ –128.



**Figure 5:** Single-GPU full-step timing vs.  $\chi_{\max}$  (companion to Table 1).

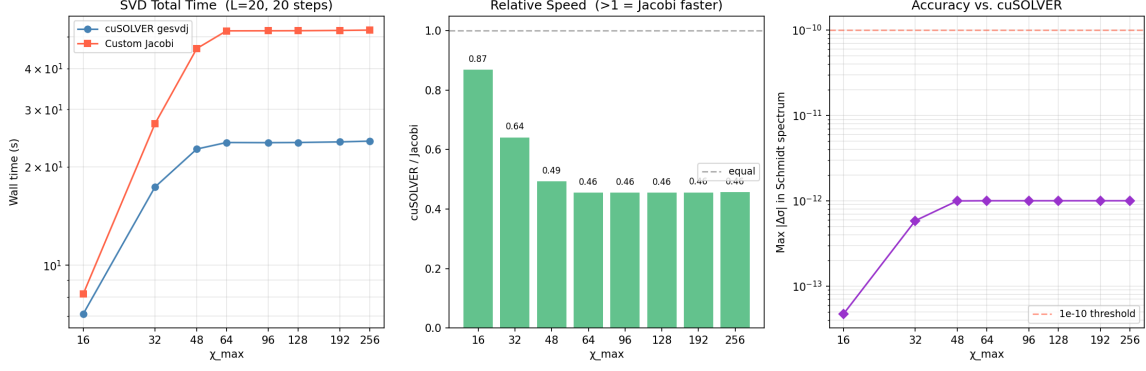
### E. Custom-kernel profile (Nsight Compute)

Nsight Compute (`ncu`) on the custom one-sided Jacobi kernel at  $\chi_{\max} = 128$  ( $256 \times 256$  complex matrix, launched `<<<1,128>>>`):

| Metric                       | Value                   | Implication                        |
|------------------------------|-------------------------|------------------------------------|
| Compute (SM) throughput      | 0.42%                   | not compute-bound                  |
| DRAM throughput              | 0.00% (10.8 MB/s)       | not bandwidth-bound                |
| L2 / L1 hit rate             | $\approx 100\%$ / 92.6% | matrix is cache-resident           |
| Achieved occupancy           | 12.5% (4/32 warps)      | one block (not a resource cap)     |
| SMs active                   | 1 of 72                 | grid is a single block             |
| Full waves across SMs        | 0.0                     | cannot fill one device wave        |
| Warp cycles per issued inst. | 22.1                    | long gaps between issues           |
| Top stall (41.6%)            | CTA barrier             | idle at <code>__syncthreads</code> |
| Avg active threads/warp      | 29.2 (18.1 pred.)       | minor lane divergence              |

**Table 5:** The custom kernel is neither compute- nor memory-bound; it is under-decomposed and latency-bound. A single 128-thread block uses  $\approx 0.17\%$  of the device (12.5% occupancy  $\times$  1/72 SMs), leaving the GPU  $\approx 99.8\%$  idle, with 41.6% of stall cycles at `__syncthreads`. The 12.5% occupancy reflects the one-block grid, not a register or shared-memory cap (theoretical occupancy is 100%).

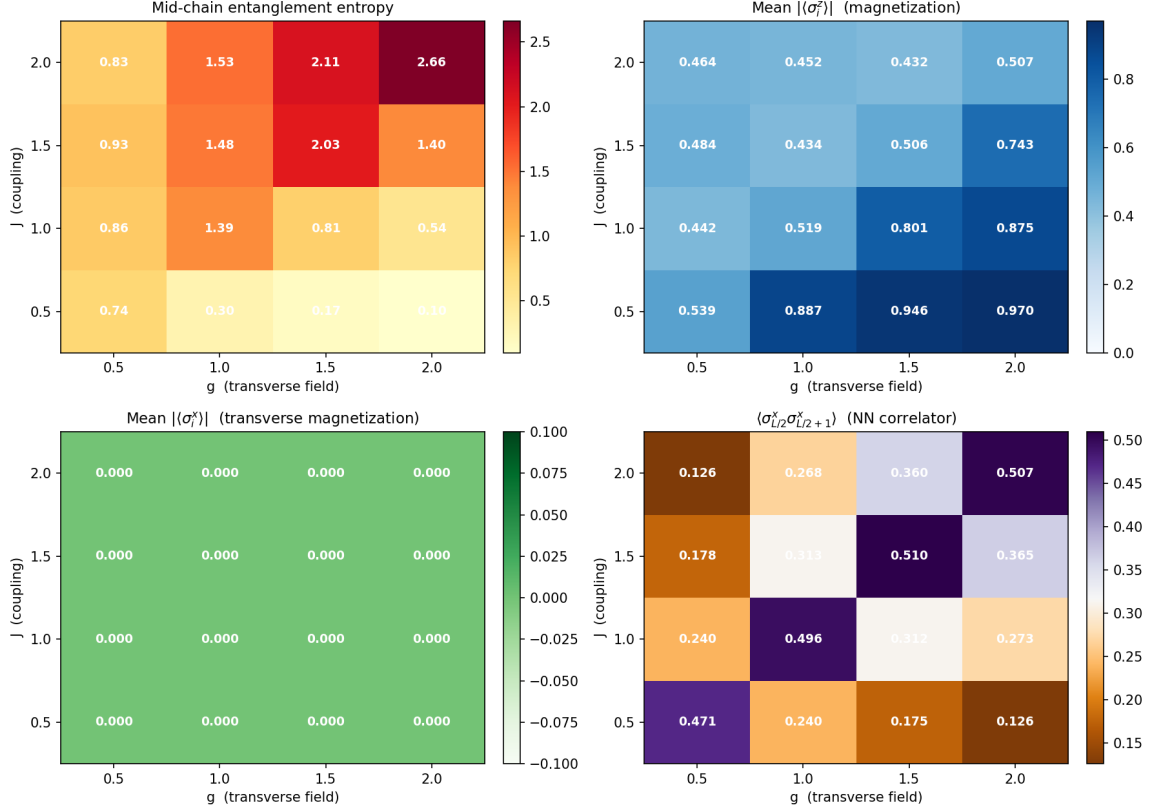




**Figure 6:** Custom Jacobi SVD vs. cuSOLVER gesvdj: time ratio (center) and Schmidt-spectrum error (right). Both plateau above  $\chi_{\max} = 64$  (companion to Table 2).

**Why the barriers dominate.** The custom Jacobi SVD carries significant synchronization overhead. For each column-pair rotation, threads first compute partial dot products and norms, then reduce them in shared memory to obtain the rotation parameters; these reductions need `__syncthreads()` barriers so that all partial sums are complete before the next reduction stage reads them. After one thread computes the rotation angle, another barrier is required before the rest of the block uses that angle to update the two columns. Since a sweep applies many such rotations sequentially and each small SVD exposes only a few warps of work, there is little independent work available to hide the barrier latency.

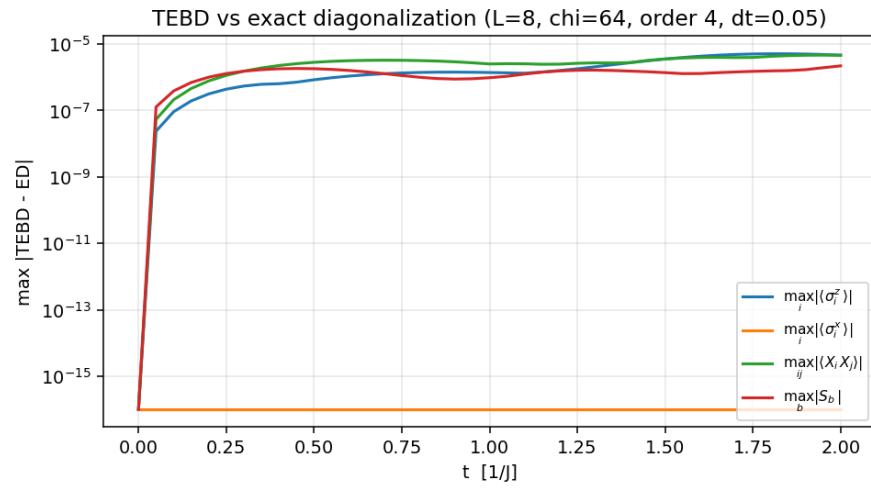
TFIM observables ( $L = 20$ ,  $\chi_{\max} = 64$ , 20 steps)



**Figure 7:** GPU-computed observables over the  $(J, g)$  phase diagram ( $L = 20$ ,  $\chi_{\max} = 64$ , 20 Trotter steps at  $dt = 0.1$ ). Top-right: mean  $|\langle\sigma_i^z\rangle|$ . Bottom-left: mean  $|\langle\sigma_i^x\rangle| = 0$  (zero by parity). Bottom-right: nearest-neighbour  $x$ -correlator.

## References

1. G. Vidal, *Efficient classical simulation of slightly entangled quantum computations*, PRL 91, 147902 (2003); *Efficient simulation of one-dimensional quantum many-body systems*, PRL 93, 040502 (2004).
2. U. Schollwöck, *The density-matrix renormalization group in the age of matrix product states*, Ann. Phys. 326, 96 (2011).
3. R. P. Brent and F. T. Luk, *The solution of singular-value and symmetric eigenvalue problems on multiprocessor arrays*, SIAM J. Sci. Stat. Comput. (1985).
4. N. Hatano and M. Suzuki, *Finding exponential product formulas of higher orders* (Suzuki–Trotter decomposition).
5. NVIDIA cuSOLVER / cuQuantum (cuTensorNet) documentation.
6. TeNPy, ITensor, quimb (open-source tensor-network libraries).



**Figure 8:** TEBD vs. exact diagonalization, per-step max error. Final fidelity loss  $4.3 \times 10^{-11}$ .